# 第三章 中尺度模式操作环境及命令

## 3.1 Linux 常用命令

### 3.1.1 Linux 简介

**历史背景** Linux 操作系统核心最早是由芬兰的 **Linus Torvalds** 1991 年 8 月在芬兰赫尔辛基大学上学时发布的，后来经过众多世界顶尖的软件工程师的不断修改和完善，Linux 得以在全球普及开来，在服务器领域及个人桌面版得到越来越多的应用，在嵌入式开发方面更是具有其它操作系统无可比拟的优势，并以每年 100%的用户递增数量显示了 Linux 强大的力量。

**系统特征** Linux 是一套免费的 32 位**多人多工**的操作系统，运行方式同 UNIX 系统很像，但 Linux 系统的稳定性、多工能力与网络功能是许多商业操作系统无法比拟的，Linux 的特色在于**源代码完全公开**，在符合 GNU GPL (General Public License) 的原则下，任何人皆可自由取得、散布、甚至修改源代码。

> “多人多工”意味着系统允许多个人员同时远程登录，并各自运行计算而不相互干扰。

**其他特色**

① 采用**阶层式目录结构**，文件归类清楚、容易管理。根目录从 / 开始，其下有 bin、boot、home 等。
② 支持**多种文件系统**，如 Ext2FS, ISOFS 以及 Windows 的文件系统 FAT16, FAT32, NTFS 等。
③ 具有**可移植性**，系统核心只有小于 10%的源代码采用汇编语言编写，其余均是采用 C 语言编写，因此具备高度移植性。
④ 可与其它的操作系统**并存**于同一台计算机上。

**串行计算** 传统软件常用，一个问题被分解为**一系列离散的指令**，这些指令**顺次执行**，所有指令只在**一个处理器**上被执行，任意时刻，最多**只有一个**指令能够被执行。

> 一般在串行计算数值模式中，格点从左下角开始，逐次运算执行。

**并行计算** 同时使用多个计算资源来解决一个计算问题，一个问题可以被分解为一系列可以并发执行的离散部分，每个部分可以被进一步分解为一系列离散指令，每个部分的指令可以在不同的处理器上被同时执行。

> 一般在并行计算数值模式中，我们将模式总体分为四块，同时从各自的左下角进行计算。

**SLURM 资源管理系统**

一种用于大型计算节点集群的作业调度系统。其维护者一个待处理工作的队列并管理此工作的整体资源利用，它以一种共享或非共享的方式管理可用的计算节点。它会为任务队列合理地分配资源，监视作业直至其完成。

在不同资源管理系统中，命令提交、查看、删除等的命令都不相同。

**SLURM** **提交任务：sbacth** 将执行的命令存在一个.sh 脚本中并提交执行。

slurm_job.sh 脚本示例

```
#!/bin/bash
#SBATCH -j test                    #作业名称
#SBATCH -P Teachq                  #资源分区
#SBATCH -N 1                       #节点数，一个节点最多 28 核
#SBATCH -ntasks-per-node=4         #申请每个节点 4 核运行
#SBATCH -o %j.out                  #标准输出文件，%j 表示输出文件为 id 的名字
#SBATCH -e %j.err                  #标准错误文件
#SBATCH -time=01:00:00             #作业最大运行时间
nprocs=$SLURM_NPROCS               #获取核数
```

```
time -p mpirun -np $nprocs ./wrf.exe      # mpirun 并行运行
```

**作业状态信息查看：squeue**　　如果只需要查看自己的任务：**squeue -u username**

> 包含有 jobid、name 任务名称、user 用户、time 为已运行时间、nodes 表示占用节点数、ST 为状态（R 为运行、PD 为等待中、E 为错误）。

**作业取消：scancel** [JOBID]

### 3.1.2 路径命令

**显示路径**　显示当前路径：**pwd**　　pwd 即 Print Working Directory 的缩写

```
>> [meso_yangc@log03 ~]$ pwd      >> /nuist/u/home/meso/meso_yangc
```

**路径概念**　**绝对路径**为：/nuist/u/home/meso/meso_yangc 绝对路径从根目录 / 开始写起，路径唯一
**相对路径**为：meso_yangc/　相对于当前你所在的文件夹进行推算的路径 .代表当前 ..代表上一层

**进入目录**　进入指定目录：**cd 目录名称**　cd 即 Change Directory 的缩写

例子：cd teacher 进入 teacher 文件夹

```
>> [meso_yangc@log03 ~]$ cd teacher/     >> [meso_yangc@log03 teacher]$
```

终端提示符的方括号内，也会同步显示当前所处的目录名。

**目录切换**　常见相对路径跳转命令与终端显示：**cd** 或 **cd ~**（进入使用者的 Home 目录）
**cd ..** 返回到上一级目录　　**cd -** 回到上一次访问的目录

```
>> [meso_yangc@log03 teacher]$ cd ..      >> [meso_yangc@log03 ~]$
>> [meso_yangc@log03 teacher]$ cd         >> [meso_yangc@log03 ~]$
```

**创建目录**　在 Linux 系统中建立新目录的命令是：　**mkdir [-m 模式] [-p 目录名] 目录**

```
>> [meso_yangc@log03 ~]$ mkdir yangchun
```

**参数说明**　**-m 模式**：在建立目录时把按模式指定设置目录权限。该目录的权限分为：目录所有者的权限、组中其他人对目录的权限和系统中其他人对目录的权限。这三个权限分别用三个数字之和来表示：对目录的**读权限是 4、写权限是 2、执行权限是 1**。

> 若设置权限为 755，即代表所有者拥有 4+2+1=7 全权限，同组人员拥有 4+1=5 读和执行权限。任何一种权限的数值都不一样，具有唯一性。

**-p 目录名**：建立所有不存在父目录的目录。

> 非常实用的参数，能够一次性创建嵌套多层的文件树，如 `mkdir -p model/data/output`。

### 3.1.3 文件命令

#### 3.1.3.1 显示文件

**显示文件**　显示指定工作目录中所包含的内容的指令是：　**ls [选项] [文件目录列表]**　（颜色反映文件类型）

> 深蓝色为目录、绿色为可执行文件、黑色为普通文件、红色为压缩包、浅蓝色为快捷方式。

**常用选项**

| | | | |
|---|---|---|---|
| **-a** | 显示所有文件及目录，包括**隐藏文件**。 | | |
| **-l** | **详细信息**：除文件名外，也将文件状态、权限、拥有者、文件大小等信息详细列出。 | | |
| **-t** | 根据文件建立时间的**倒序次序**列出。 | | |
| **-help** | 显示帮助信息 | **-X** | 按扩展名排序显示 |
| **-R** | 递归显示下层子目录 | **-version** | 显示版本信息 |
| **-c** | 按列输出，纵向排序 | **-x** | 按列输出，横向排序 |
| **-A** | 同 -a，但不列出 .（目前目录）及 ..（父目录） | | |

**权限解读** 使用 ls -l 查看文件详细状态时，会输出如下格式的内容：

```
>> [meso_yangc@log03 ~]$ ls -l
>> drwxr-xr-x        2    meso_yangc    meso     4096     3 月 17 05:54     student
>> drwxr-xr-x        2    meso_yangc    meso     4096     3 月 17 04:50     teacher
>> drwxr-xr-x        2    meso_yangc    meso     4096     3 月 17 06:06     yangchun
```

针对首列的 10 个字符 drwxr-xr-x：**第 1 列为文件类型**，第 2-4 列为**该文件所有者**的权限，第 5-7 列为**组中其他人**的权限，第 8-10 列为**系统中其他人**的权限。

**文件类型**：**d 代表文件夹**，l 软连接，**- 普通文件**，b 设备文件中的可供存储的接口设备，c 设备文件中的串行端口设备（如键盘鼠标）。

**权限字符**：权限由三个字符组成，依次表示对文件的**读**（用字符 r 表示）、**写**（用字符 w 表示）和**执行权限**（用字符 x 表示）。当用户没有相应的权限时，该权限的对应位置用短线 - 来表示。

例如：drwxr-x--- 表示的含义是：d 表示这条信息是目录；目录拥有者的权限是 rwx（表示有读、写和执行权限）；组中其他人对该目录的权限是 r-x（表示有读和执行权限，没有写权限）；系统中其他人对该目录的权限是 ---（没有任何权限）。

气象模式运行时常需要读写大量文件，如果遇到 Permission denied 报错，通常就是此处的写权限 w 或执行权限 x 没有正确配置。可以使用 **chmod** 修改文件的权限，例如 `chmod 777 finaname`。

此外，如果想将脚本转换为仅对用户本人有执行权限，可以使用：`chmod u+x finaname`。

### 3.1.3.2 查找文件

**查找文件** 在 Linux 系统中查找文件的命令通常为 find 命令。find 命令可在使用、管理 Linux 系统中方便地查找所需要的指定文件。 find 命令的语法格式如下：**find** [目录列表] **[匹配标准] [查找完执行的 action]**

**目录列表**：希望查询文件或文件集的目录列表，目录间用空格分隔。

**匹配标准**：希望查询的文件的匹配标准或说明。详细的匹配标准如下表所示。

**参数列表**

| | |
|---|---|
| **-name filename** | 找出**文件名为 filename** 的文件 |
| **-print** | 显示**整个文件路径和名称**。一般来说，都要用 -print，如果没有这个参数，则 find 命令进行所要求的搜索是没有显示结果的。 |
| -perm mode | 查找文件权限刚好等于 mode 的文件，mode 用数字表示，如 755。<br>不仅仅是读，写和执行，所有模式都必须匹配。如果在模式前是 - ，表示采用除这个模式外的所有模式。 |
| -type type | 查找文件的类型为 TYPE 的文件，TYPE 的值主要有：一般文件 (f)、设备文件 (b、c)、目录 (d)、连接文件 (l)、socket (s)、FIFO 管道文件 (p)。 |
| -links n | 匹配所有连接数为 n 的文件。 |
| -user 用户号 | 匹配所有用户序列号是前面所指定的用户序列号的文件，可以是数字型的值或用户登录名。 |
| -atime n | 匹配所有在前 n 天内访问过的文件。 |
| -mtime n | 匹配所有在前 n 天内修改过的文件。 |
| -newer 文件 | 匹配所有修改时间比 file 文件更新的文件。 |
| -size [+/-]size | 找出比 SIZE 还要大 (+) 或小 (-) 的文件。 |

**通过文件名查找**

知道了某个文件的文件名，却不知道它存于当前目录下的哪个目录下，此时可通过查找命令找到该文件。

```
>>[meso_yangc@log03 ~]$ find . -name teacher.log        ./teacher/teacher.log
```

**根据部分文件名查找**

当要查找某个文件时，不知道该文件的全名，只知道这个文件包含几个特定的字母，此时用查找命令也是可找到相应文件的。这时在给字查找文件名时通配符 *（任意字符）、?（单个字符）。

例如，还是查找文件 namelist.input，但只记得文件名包含 namelist 字符串，可使用命令：

```
>>[meso_yangc@log03 ~]$ find . -name “namelist*” -print
```

**查找内容** **查找文件内容**的命令是 **grep** 命令。**grep [选项] 匹配字符串 文件列表**

**参数说明**

- **-n** 每个匹配行只按照**相对的行号**显示。
- **-i** 产生**不区分大小写**的匹配(缺省状态是区分大小写)。例如 namelist.input 中都是小写的内容，运行时产生的 namelist.output 中都是大写的内容。
- -v 列出不匹配串或正则表达式的行。
- -c 对匹配的行计数。
- -l 只显示包含匹配的文件的文件名。
- -h 抑制包含匹配文件的文件名的显示

**查找文件内容实例**

```
>> [meso_yangc@log03 teacher]$ grep moon teacher.log
>> The sun, the moon and you. The moon for the night
>> [meso_yangc@log03 teacher]$ grep moon *
>> teacher2.log:The sun, the moon and you.
>> teacher2.log:The moon for the night
>> teacher.log:The sun, the moon and you.
>> teacher.log:The moon for the night
```

grep 在模式排错中是非常有用的。当模式报错中断时，我们通常不用打开几百兆的庞大日志文本逐行查看，而是直接使用 `grep -i "error" rsl.error.*` 命令，就能定位出所有报错节点和错误信息行。

### 3.1.3.3 文件复制、移动与删除

**文件复制** Linux 下的 cp 命令用于复制文件或目录，该命令是最重要的文件操作命令。

**cp [选项] 源文件 目标文件**： 将源文件**拷贝并重命名**到目标文件

**cp [选项] 源文件组 目标目录**： 将源文件组拷贝到目标目录中（添加 -r 参数以递归拷贝）

**常见选项**

- **-r** **递归拷贝目录**，把所有非目录文件当普通文件拷贝。
- -R 递归拷贝目录。
- -a 在备份中保持尽可能多的源文件结构和属。
- -b 作将要覆盖或删除文件的备份。
- -f 删除已存在的目标文件。
- -i 提示是否覆盖已存在的目标文件。
- -p 保持原先文件的所有者，组权限和时间标志

```
>>[meso_yangc@log03 ~]$ ls            >> student   teacher  yangchun
>>[meso_yangc@log03 ~]$ cp teacher/teacher.log ../march.log     拷贝文件并重命名为 march.log
>>[meso_yangc@log03 ~]$ ls            >>student    teacher  march.log    yangchun
```

**删除操作** 文件及目录删除。rm 命令可从文件系统中删除文件及整个目录：**rm [选项] 文件名**

**常用选项**

- **-r** 删除文件列表中**指定的目录（递归执行）**，若不用此标志则不删除目录
- **-i** 指定交互模式，在删除前**提示确认**。任何以 Y 开始的响应都表示肯定，其他则表示否定。
- **-f** 指定**强行删除**模式。通常，在删除文件权限可满足时 rm 提示。本标志强迫删除，不用提示。
- **-v** 在删除前回显文件名。
- **--** 指明所有选项结束。用于删除一个文件名与某一选项相同的文件。

rm -rf 会强制且递归地删除指定目录下的所有文件，不会有任何提示，是 Linux 中最危险的命令之一。在清理气象模式的废弃数据时，务必仔细核对目标路径。

```
>>[meso_yangc@log03 teacher]$ ls
>> teacher2.log      teacher.log      teacher.txt
>>[meso_yangc@log03 teacher]$ rm teacher.txt
>>[meso_yangc@log03 teacher]$ ls
>> teacher2.log      teacher.log
```

**移动改名** ① 在 Linux 系统中，移动文件或目录可使用 **mv** 命令：**mv [参数] 原文件位置 目标文件位置**
② mv 还可重命名，把源文件以一个新文件名称移动到新的目录中去：**mv [参数] 原文件 新文件名**

移动文件或目录：`mv file1 /home/cyang/` 给文件或目录改名：`mv file1 file2`

**参数选项**
-f 强制的意思，强制直接移动而不询问；
-I 若目标文件已经存在时，就会询问是否覆盖
-u 若目标文件已经存在，被移动文件更新时间较原存在文件新，才会更新

mv 即 move 的缩写。在重命名文件或整理目录结构时非常有用。例如将 namelist.input.test 重命名为模式所需的 namelist.input。

**创建文件** 创建一个一般的普通文件，并且这个文件为空。**touch 文件名**

例如：touch teacher.txt

```
>>[meso_yangc@log03 teacher]$ ls                    >>teacher2.log   teacher.log
>>[meso_yangc@log03 teacher]$ touch teacher.txt
>>[meso_yangc@log03 teacher]$ ls                    >>teacher2.log   teacher.log      teacher.txt
```

touch 除了创建空文件外，还可以用于修改现有文件的时间戳。在一些气象脚本中，可能会用 touch 生成一个占位符文件，用于标记某个计算流程已经完成，但这并不常用，我们一般使用 vi 命令。

### 3.1.3.4 文件比较

**文件比较** diff 命令用于比较两个文件内容的不同，其命令语法格式如下：**diff [参数] 原文件 目标文件**
其中源文件和目标文件是用户要比较的两个文件。

**参数选项**
-a 将所有文件当做文本文件来处理
-b 忽略空格造成的不同
-B 忽略空行造成的不同
-q 只报告什么地方不同，不报告具体的不同信息
-H 利用试探法加速对大文件的搜索
-i 忽略大小写的变化
-l 用 pr 对输出进行分页
-r 在比较目录时比较所有的子目录
-s 两个文件相同时才报告
-v 在标准输出上输出版本信息并退出

实例演示

```
>>[meso_yangc@log03 teacher]$ diff teacher.log teacher2.log
1c1
< this is the first file
---
> this is the second file
```

diff 命令在比对气象模式的参数设置文件（如 WRF 的 namelist.input）时非常实用，能快速找出两次不同实验中究竟修改了哪些物理参数。

### 3.1.4 链接与压缩命令

包含文件权限修改、链接创建以及文件的打包压缩

**链接命令** 为某一个文件在另外一个位置建立一个同步的链接。

**使用方法** ① **软链接**：**ln -s 原文件 链接文件** 生成一个文件的镜像，不会占用磁盘空间。

② **硬链接**：**ln 原文件 链接文件**（没有参数-s） 生成一个和源文件大小相同的文件

软链接时，只能从其他目录镜像文件到本目录，不能链接当前目录。

**命令选项** **-s 进行软链接**

**-f** 链接时先将与目标位置同名的档案删除。

**实例演示**

```
>>[meso_yangc@log03 ~]$ ls                                  >>student teacher yangchun
>>[meso_yangc@log03 ~]$ ln -sf teacher/teacher.log .        >> teacher.log -> teacher/teacher.log （成功）
>>[meso_yangc@log03 ~]$ ls                                  >>student teacher teacher.log yangchun
```

在中尺度气象模式中，软链接 ln -s 是最常用的命令之一。比如在运行 real.exe 前，需要将 WPS 生成的巨大气象强迫场文件（met_em*）链接到 WRF 运行目录。如果不使用软链接而使用复制，会瞬间耗尽服务器的存储空间。

**文件打包** **tar** 可以对文件和目录进行打包。利用 tar，用户可以对某一特定文件进行打包，也可以在包中改变文件，或者向包中加入新的文件。**tar [主选项+辅选项] 文件或者目录**

**压缩格式** **tar cvf *.tar 需要打包的文件或目录**

**解压缩格式** **tar xvf *.tar** (解压.gz 文件使用 zxvf)

**命令选项**

| | |
|---|---|
| **-c** | **创建**新的档案文件。 |
| **-x** | 从档案文件中**释放**文件。 |
| **-v** | 详细报告 tar 处理的**文件信息**。 |
| **-f** | 使用档案文件或设备，这个选项通常是**必选**的。 |
| **-z** | 用 **gzip** 来压缩/解压缩文件 |
| -r | 把要存档的文件追加到档案文件的末尾。 |
| -t | 列出档案文件的内容，查看已经备份了哪些文件。 |
| -u | 更新文件。 |
| -K | 保存已经存在的文件 |
| -m | 在还原文件时，把所有文件的修改时间设定为现在。 |

- 解压文件后缀：*.tar.gz

```
>>[meso_yangc@log03 teacher]$ ls          >>teacher2.log
>>[meso_yangc@log03 teacher]$ tar zcvf teacher.tar.gz teacher2.log      >>teacher2.log
>>[meso_yangc@log03 teacher]$ ls          >>teacher2.log teacher.tar.gz
>>[meso_yangc@log03 teacher]$ mv teacher2.log teacher.log
>>[meso_yangc@log03 teacher]$ ls          >>teacher.log teacher.tar.gz
>>[meso_yangc@log03 teacher]$ tar zxvf teacher.tar.gz                   >>teacher2.log
>>[meso_yangc@log03 teacher]$ ls          >>teacher2.log teacher.log teacher.tar.gz
```

-f 必须放在选项的最后一位，紧挨着压缩包的文件名，例如 tar -cvf backup.tar ./data。
如果解压.gz 文件，还需要增加参数为：tar zcvf 或 tar zxvf

**修改权限** 重新设置文件的权限，使用的命令是 **chmod** 命令。并不是谁都可改变文件和目录的访问权限，只有文件和目录的所有者才有权限修改其权限，另外超级用户可对所有文件或目录进行权限设置。

chmod 命令的使用语法格式如下：**chmod [选项] 文件和目录列表**

**命令选项**

**-c**：只有在文件的权限确实改变时才进行详细说明

**-f**：不打印权限不能改变的文件的错误信息

**-v**：详细说明权限的变化

**-R**：递归改变目录及其内容的权限

**权限设定** ① **使用字符串设置权限**

例如：chmod u+rwx file1

```
>>[meso_yangc@log03 teacher]$ ls -l teacher2.log
>>-rw-r--r-- 1 meso_yangc meso 133 3月 17 06:58 teacher2.log
>>[meso_yangc@log03 teacher]$ chmod u+x teacher2.log
>>[meso_yangc@log03 teacher]$ ls -l teacher2.log
>>-rwxr--r-- 1 meso_yangc meso 133 3月 17 06:58 teacher2.log
```

② **使用八进制数设置权限**：另外，文件和目录的权限还可用八进制数字模式来表示。三个八进制数字分别代表 ugo 的权限。读权、写权和执行权所对应的数值分别是 4，2 和 1。

案例

```
>>[meso_yangc@log03 teacher]$ ls -l teacher2.log
>>-rwxr--r-- 1 meso_yangc meso 133 3月 17 06:58 teacher2.log
>>[meso_yangc@log03 teacher]$ chmod 755 teacher2.log
>>[meso_yangc@log03 teacher]$ ls -l teacher2.log
>>-rwxr-xr-x 1 meso_yangc meso 133 3月 17 06:58 teacher2.log
```

> 气象科研人员经常需要自己编写 Shell 或 Python 脚本自动化处理数据。新创建的脚本默认是没有执行权限的，运行 chmod +x script.sh 或 chmod 755 script.sh 是让脚本跑起来的必经步骤。

**文件压缩** **gzip 压缩文件**。gzip 命令是在 Linux 系统中经常使用的一个对文件进行压缩和解压缩的命令，既方便又好用。语法格式如下：gzip [选项] 压缩（解压缩）的文件名

**命令选项**

-d 将压缩文件解压。

-l 对每个压缩文件，显示压缩文件的大小、未压缩文件的大小、压缩比、未压缩文件的名字等详细信息。

-r 递归式地查找指定目录并压缩其中的所有文件或者是解压缩。

-t 测试，检查压缩文件是否完整。

-v 对每一个压缩和解压的文件，显示文件名和压缩比。

**文件解压** **unzip 命令**。可以用于解扩展名为 .zip 的压缩文件。语法格式如下：unzip [选项] 压缩文件名.zip

**命令选项**

-x 文件列表 解压缩文件，但不包括指定的 file 文件

-v 查看压缩文件目录，但不解压。

-t 测试文件有无损坏，但不解压。

-d 目录把压缩文件解到指定目录下。

-z 只显示压缩文件的注解。

-n 不覆盖已经存在的文件。

-o 覆盖已存在的文件且不要求用户确认。

-j 不重建文档的目录结构，把所有文件解压到同一目录下。

# 3.2 文本编辑器

## 3.2.1 Vi 概述

### 3.2.1.1 基本内容和工作方式

**各类编辑器** Linux 下有很多的文字编辑器，例如 emacs, pico, nano, joe, **vi/vim** 等。

**为什么气象科研一定要学习 vi/vim**

所有的类似 Unix 系统都会内建 vi/vim 文书编辑器，其它编辑器不一定存在。别的软件会主动呼叫 vi/vim。vi/vim 具有程序编辑的能力，可以主动的以字体颜色辨别语法的正确性，方便程序设计。而且编辑速度较快。

**vi 的特点** vi 是 **visual interface** 的简称，是交互式的全屏幕编辑程序。其主要特点包括：

① 文本编辑器（没有排版功能，处理纯文本字符）。

② 全屏幕编辑（从行编辑发展过来）。

③ **命令模式**，不提供菜单，与 dos 提供的 edit 有很大的差异。

**区别** ① **vi** 是一款基础的文书处理器。

② **vim** 是 vi 的升级版，在 vi 的基础上能显示特殊的信息（如代码高亮等）。

**工作方式** vi 有三种主要工作模式：**命令方式、输入方式、末行方式**。

① **命令方式**：用户一进入 vi 就进入了命令方式，在该模式中任何键入的字符都被看成 **vi 的命令**，键入后立即执行。

② **输入方式**：当用户需要输入文本时，使用**某个命令**，进入**输入方式**，才可开始输入文本。

③ **末行方式**：也称 **ex 模式**，在命令方式中键入**冒号 ：**，就进入了**末行方式**，在该模式中输入 ex 中的命令，例如 w（写）和回车，就可将编辑的内容存入文件。

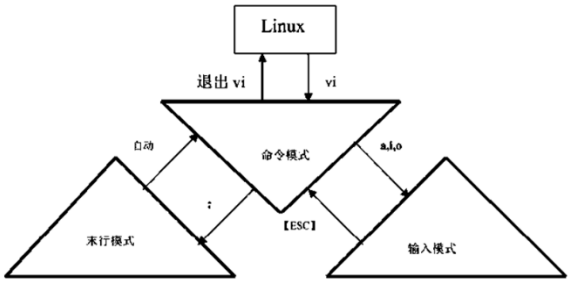
**方式转换** ① 在操作系统提示符下键入 vi，就进入了**命令方式**。

② 进入**输入方式**，键入**编辑命令**，如插入命令 **i**、添加命令 **a**、建立新行 **o** 等（可小写可大写，作用不同）。

③ 进入**末行方式**，只需键入**冒号：后面跟上命令**，每次只执行一条，执行完毕立即回到命令方式。

④ 不论在什么方式，只要按 **ESC** 键，就回到**命令方式**。

⑤ 输入方式和末行方式之间不能直接切换，必须通过命令方式。




⑥ 退出 vi 的方式有：**命令方式**，用两个 **ZZ**（大写）；**末行方式**键入 **q** 或 **q! (不保存)**或 **wq(保存)**。

初学者最容易犯的错误就是在输入模式下按快捷键保存，导致文本里打出了一堆无意义的字符。任何保存或退出操作前，先按几次 ESC 键回到命令模式，这是最保险的做法。

### 3.2.1.2 进入和退出 Vi

**进入 vi** ① **vi filename** 这是编辑一个新文件或**编辑一个已经存在的文件**的方法。

如果是新文件，系统会为你创建一个名为 filename 的文件。

如果你需要对一个存在的文件进行修改，系统会把文件的内容读入 vi 使用的缓冲区，供你进行编辑，完成编辑后，用相关命令写入原文件或写入另一个新的文件。

② **vi** newfile 这是**编辑一个新文件**的方法。在编辑完成后必须用相关命令写入一个新的文件，否则一切编辑动作都无效。

**光标定位** 进入 vi 后，如果是新文件光标会停留在文件的**第一行行首**。如希望停留在其他行，可用下面的命令：

① vi **+n** filename：进入 vi 后当前光标位置为第 n 行。

② vi **+** filename：进入 vi 后当前光标位置为文件尾。

③ vi **+/string** filename：进入 vi 后当前光标位置为字符串 string 所在行的行首。

在调试 WRF 模式代码时，如果编译器报错指出 module_first_rk_step_part1.F 的第 1500 行有语法错误，直接使用 vi +1500 filename 能够瞬间跳转到报错位置，极大提升排错效率。

**退出 vi**　① 以 vi filename 方式进入 vi，并保存已经作过的编辑，退出 vi：
先 ESC 退出输入方式，进入命令方式。
a. 在命令模式下退出，用两个大写的 **ZZ** 即可。
b. 在末行方式下退出，用以下命令：　**:wq** 或 :x
② 以 vi 方式进入 vi 模式，进行了内容编辑，**退出时必须进入末行方式**，用以下命令：
先 **:w newfilename**（保存到新文件）　再 **:q**（退出）
③ 不保存所作过的一切编辑动作，在末行方式下退出 vi，直接输入 **:q!**

> 如果在编辑气象配置文件时，你不小心删错了重要参数且不知如何恢复，千万不要按 :wq 保存。直接输入 :q! 强制退出不保存，然后重新打开文件即可。

### 3.2.2 移动光标

**移动模式**　移动光标（在命令方式下进行，不要在输入模式下按这些键，否则会变成输入字符）。

**显示行号**　vi 的工作是在文本缓冲区中做的，为了帮助用户观看，可为文本的行设置一个行号，放在文本的左侧（行首前）。可以用末行命令 set 来设置和取消：
① **:set number** 显示行号　② :set nonumber 取消显示行号

**单位定义**

在学习光标移动快捷键前，需要先了解 vi 是如何定义光标移动单位的：
① **字符**：汉字由二个 ASCII 字符宽度，移动的单位是一个汉字，英文或数字等是一个 ASCII 字符宽度。
② **行**：文本的一行，可用行号定位。
③ **屏幕**：分成半个屏幕高或整个屏幕高（页）。
④ **行首和行尾**：文本行的行尾和行首。
⑤ **字**：分为广义字和狭义字。广义字：由空格作为间隔符。狭义字：英文单词及一些特殊字符均作为一个字。
⑥ **句**：以逗号、句号、问号、感叹号结尾且后面有二个以上（含二个）空格或一个换行符的字符序列。
⑦ **段**：以一个空白行开始和结束的字符序列。

**基础移动**　**按字符移动的键**：
① **四个带箭头的方向键**：各自按方向移动一个字符。
② k、j、h、l：各自按上、下、左、右移动一个字符。
③ **spacebar**：光标右移一个字符。　**backspace**：光标左移一个字符。
**ctrl+n**：光标下移一个字符。　**ctrl+p**：光标上移一个字符。

**行级移动**　**按行来移动光标**：
① 0：行首　② ^：第一个非空白字符
③ $：光标移动到本行行尾　④ -：光标移动到上一行的行首
**跳到指定的行**：**[行号]G**　缺省为到文件的最后行。

> 非常常用的快捷键！例如 5G 表明移到第 5 行，1G 移到第一行，单独按 G 移到最末行。

**字级移动**　**按字移动光标：**（小写字母对狭义的字，大写字母对广义的字）
① w 和 W：将光标移动到下一个字的字首。
② e 和 E：将光标移动到下一个字的字尾。
③ b 和 B：将光标移动到前一个字的字首。

> 例子：在单词间快速跳跃，如从 "time" 跳到 "you" 即可用 w。

**翻页滚动**　① ctrl+u：将屏幕向文件头方向翻滚半屏。
② ctrl+d：将屏幕向文件尾方向翻滚半屏。
③ **ctrl+f**：将屏幕向**文件尾**方向**翻滚一整屏** (forward)。
④ **ctrl+b**：将屏幕向**文件头**方向**翻滚一整屏** (backward)。

> 查看长达几千行的气象数据日志时，用 Ctrl+F 和 Ctrl+B 翻页比狂按向下箭头要高效得多。

### 3.2.3 文本操作

#### 3.2.3.1 文本的插入

**模式切换** 输入任意文本，都要将 vi 的工作方式从命令方式切换到**文本输入方式**。

**命令列表** **如下命令进入文本输入方式**（注意当前光标位置的相对变化）:

① **i** 插入**当前字符前**（插入模式） ② **I** 插入**当前行头**

③ a 插入**当前字符后** ④ A 插入当前行**尾**

⑤ o 插入**当前行下一行**（新的行） ⑥ O 插入当前行**上一行**

> 小写通常是相对当前字符，大写通常是相对当前行。在修改气象模式配置文件（如 namelist.input）时，常用 A 快速跳转到行尾修改参数值，用 o 新起一行添加新的参数项。

#### 3.2.3.2 文本的删除

> 此处的命令都是命令模式下的。

**字符删除** ① **x** 删除**光标处的**字符 ② nx 删除光标位置起的右 n 个字符

③ **X** 删除**光标前的**字符 ④ nX 删除光标位置前的左 n 个字符

**行级删除** ① dd 删除当前行 ② ndd 删除当前行起的 n 行

**部分删除** ① D 删除光标起到行尾的内容 ② d0 删除光标前一个字符到行首的内容

**单词删除** ① dw 删除一个单词 ② ndw 删除 n 个单词

> 这里的 n 代表数字，体现了 vi 的高效性。例如在清理错误的气象站数据行时，用 10dd 可以瞬间删掉连续的 10 行废数据。

#### 3.2.3.3 恢复删除与修改文本

**撤销恢复** ① **u** **取消上一次的编辑动作**(可多次) ② U 取消在本行所有的编辑动作

③ 重复命令(.) 再一次执行刚执行过的命令

**文本修改** 修改文本结合了删除与进入插入模式的双重动作。

① C 改变光标位置起到行尾的内容 ② nC 改变光标位置起的 n 行内容

③ cc 改变当前行的内容 ④ ncc 改变当前行起 n 行的内容

#### 3.2.3.4 文本替换

**替换概念** 文本替换是用新的文本替换原有的文本。

**字符替换** ① **rx**: 用 x 字符替换光标位置的字符 ② nrx: 用 x 字符替换光标位置起的 n 个字符

③ **R**: 用跟随其后的字符一对一地替换文本,结束后应用【ESC】键回到命令模式

④ nR: 表示 R 后面输入的内容重复 n 次

#### 3.2.3.5 文本行移动

**文本行移动** 实质是先把要移动的 n 行先删除,然后粘贴到目的行。

**移动步骤** ① 将光标移到要移动的 n 行首行 ② **ndd** 命令: 删除当前起的 n 行

③ 将光标移到目的行处 ④ 键入 **p**，将被删除的 n 行粘贴到当前行

**末行命令** 使用末行命令 **m** 移动命令: `n1,n2 m k` (将从 n1 行到 n2 行的文本移动到 k 行下,其中 m 是移动命令)

#### 3.2.3.6 拷贝文本

**命令拷贝** 第一步: **拷贝** 拷贝一行(从当前行) **yy** 拷贝光标所在字 : yw

第二步: **粘贴** 将光标移到目标位置，按命令 p。

**末行拷贝** 末行方式: 用 **co 命令** 一步即可。

> 例: `:1,5 co 20` 将 1 到 5 行拷贝到 20 行下
> 例: `:'a,'b co` 将 ab 标识行拷贝到当前行下

# 3.3 Shell 脚本

## 3.3.1 Shell 的本质与背景

**背景信息** Windows、Mac OS 等系统，都是带**图形界面**的，简单直观，容易上手。计算机的早期并没有图形界面，通过**命令行**来控制计算机。图形界面和命令行是架设在用户和操作系统内核之间的一座桥梁。

**Shell 的本质** 在 Linux 下，这个命令行程序叫做 **Shell**。Shell 是一个**应用程序**，它连接了用户和 Linux 内核，让用户能够更加高效、安全、低成本地使用 Linux 内核，这就是 Shell 的本质。

## 3.3.2 常见 Shell 解释器

**Shell 种类** Shell 既是一个连接内核和用户的软件，也是一种脚本编程语言，常见的 Shell 有 **sh、bash、csh、tcsh、ash** 等。

**sh**：sh 的全称是 Bourne shell。sh 是 UNIX 上的标准 shell，很多 UNIX 版本都配有 sh。

**csh**：语法有点类似 C 语言，所以才得名为 C shell，简称为 csh。

**tcsh**：tcsh 是 csh 的增强版，加入了命令补全功能，提供了更加强大的语法支持。（大公司一般用 tcsh）

**ash**：一个简单的轻量级的 Shell，占用资源少，适合运行于低内存环境。

**bash**：**bash shell** 是 Linux 的默认 shell，保持了对 sh shell 的兼容性。bash 兼容 sh 意味着，针对 sh 编写的 Shell 代码可以在 bash 中运行。

## 3.3.3 Shell 提示符的含义

**提示符拆解** 终端上形如 `[meso_yangc@log03 student]$` 各个部分的含义如下：

① [ ] 是提示符的分隔符号，没有特殊含义。

② meso_yangc 表示当前**登录的用户**。

③ @ 是分隔符号，没有特殊含义。

④ log03 表示当前系统的**简写主机名**。

⑤ student 代表用户当前所在的目录，如果是 ~ 代表用户当前所在的目录为主目录（home 目录）。

⑥ $ 是命令提示符。Linux 用这个符号标识登录的用户权限等级：如果是超级用户（root 用户），提示符就是#；如果是普通用户，提示符就是$。

## 3.3.4 脚本示例 (test.sh)

脚本结构 test.sh

```
#!/bin/bash                # 解释器
echo "Hello World !"       #这是一条语句
```

**第 1 行** `#!` 是一个约定的标记，它告诉系统这个脚本需要什么**解释器**来执行，即使用哪一种 Shell。后面的 `/bin/bash` 就是指明了解释器的具体位置。

**第 2 行** `echo` 命令用于向标准输出文件（一般就是指显示器）输出文本。在 .sh 文件中使用命令与在终端直接输入命令的效果是一样的。

`#` 及其后面的内容是**注释**。Shell 脚本中所有以 `#` 开头的都是注释（当然以 `#!` 开头的除外）。

## 3.3.5 变量读取与输出

交互命令：输入输出

```
1  #!/bin/bash
2  # Copyright (c) http://c.biancheng.net/shell/
3  echo "What is your name?"
4  read PERSON
5  echo "Hello, $PERSON"
```

**第 5 行**　表示从终端**读取**用户**输入**的数据，并赋值给 PERSON 变量。read 命令用来从标准输入文件（Standard Input，stdin，一般就是指键盘）读取用户输入的数据。

**第 6 行**　表示输出变量 PERSON 的内容。注意**在变量名前边要加上$**，否则变量名作为字符串的一部分处理。

### 3.3.6 Shell 脚本的运行方式

**运行方式 1**　将 Shell 脚本**作为程序运行**。Shell 脚本也是一种解释执行的程序，可以在终端直接调用（需要使用 chmod u+x test.sh 命令给 Shell 脚本加上执行权限）。

```
>>[meso_yangc@log03 student]$ vi teacher.sh            新建一个文件
>>[meso_yangc@log03 student]$ chmod u+x teacher.sh     赋予权限（相当重要）
>>[meso_yangc@log03 student]$ ./teacher.sh             运行脚本
>>Hello World
```

**运行方式 2**　将 Shell 脚本**作为参数传递给 Bash 解释器**。直接运行 Bash 解释器，将**脚本文件的名字**作为**参数**传递给 Bash，**不需要**在脚本文件的第一行指定解释器信息，如下所示：

```
>>[meso_yangc@log03 student]$ bash teacher.sh
```

### 3.3.7 变量的定义规范

**变量定义**　Shell 支持以下三种定义变量的方式：① **variable=value** ② **variable='value'** ③ **variable="value"**
注意，**赋值号=的周围不能有空格**，value 不包含任何空白符

> 不同于 Python 或 Fortran，在 Shell 里写 a = 1 是会报错的，必须紧挨着写成 a=1。

**命名规范**　Shell 变量的命名规范和大部分编程语言都一样：变量名由**数字、字母、下划线**组成；必须以字母或者下划线开头；不能使用 Shell 里的关键字（通过 help 命令可以查看保留关键字）。

### 3.3.8 变量的使用与引号区别

**变量调用**　使用一个定义过的变量，只要在变量名**前面加美元符号 $** 即可。

```
author="严长生"
echo $author
echo ${author}
```

**注意**　变量名外面的**花括号 {}** 是可选的，加不加都行，加花括号是为了帮助解释器识别变量的边界。
推荐给所有变量加上花括号{}，这是个良好的编程习惯。

**引号区别**　**单引号与双引号**：变量的值可以由单引号 ' 包围，也可以由双引号 "" 包围，但如果**变量**中包含了其他变量的赋值，则应该使用**双引号**。

程序

```
1   #!/bin/bash
2   url="http://c.biancheng.net"
3   website1='C 语言中文网：${url}'      单引号
4   website2="C 语言中文网：${url}"      双引号
5   echo $website1                      >> C 语言中文网：${url}
6   echo $website2                      >> C 语言中文网：http://c.biancheng.net
```

> 单引号是死脑筋，双引号是活脑筋。单引号里的所有字符都会原样输出，双引号则允许解析里面的变量。我们在用 Bash 调用 awk 处理气象台站文本数据时，经常需要巧妙结合这两种引号。
>
> 从颜色上也可以分辨出，哪些是变量，哪些是变量的值。命令是黄色的、字符串是红色的、变量引用是紫色的、变量赋值是蓝色的。

### 3.3.9 传递参数（位置参数）

**传参机制**　运行 Shell 脚本文件时我们可以给它传递一些参数，这些参数在脚本文件内部可以使用 $n 的形式来接收，例如，$1 表示第一个参数，$2 表示第二个参数，依次类推。

**示例脚本** test.sh

Bash
#!/bin/bash
echo "Language: $1"
echo "URL: $2"

终端执行：

Bash
[meso_yangc@log03 student]$ ./test.sh Shell http://c.biancheng.net/shell/
Language: Shell
URL: http://c.biancheng.net/shell/
其中 Shell 是第一个位置参数，http://c.biancheng.net/shell/ 是第二个位置参数，两者之间以**空格**分隔。

> 这是实现气象自动化的基石。例如写好一个画图脚本，运行时传入 ./plot_T2m.sh 2023 07，脚本内用 $1 接收年份，$2 接收月份，就能实现任意日期的自动绘图，不用每次都去修改代码内部的值。

### 3.3.10 数组的定义与使用

**数组定义**　在 Shell 中，用**括号 () 来表示数组**，数组元素之间用**空格**来分隔
**array_name=(ele1 ele2 ele3 ... elen)**　注意，赋值号=两边不能有空格，必须紧挨着

**访问元素**　获取数组元素的值，一般使用下面的格式：**${array_name[index]}**
使用 @ 或 * 可以获取数组中的所有元素，下标 index 从零开始。

代码示例

```
Bash
#!/bin/bash
nums=(29 100 13 8 91 44)
echo ${nums[@]}          #输出所有数组元素        >> 29 100 13 8 91 44
nums[10]=66              #给第 10 个元素赋值（此时会增加数组长度，前面的位置留空）
echo ${nums[*]}          #输出所有数组元素        >> 29 100 13 8 91 44 66
echo ${nums[4]}          #输出第 5 个元素         >> 91
```

### 3.3.11 Shell 算术运算符与 (()) 命令

**算术运算符**

| | |
|---|---|
| + - | 加法、减法 |
| * / % | 乘法、除法、**取余**（取模） |
| ** | **幂运算** |
| ++ -- | **自增**和**自减**，可以放在变量的前面也可以放在变量的后面 |
| ! && \|\| | 逻辑非（取反）、逻辑与（and）、逻辑或（or） |
| < <= > >= | 比较符号（小于、小于等于、大于、大于等于） |
| == != = | 比较符号（相等、不相等；对于字符串，= 也可以表示相当于） |
| << >> | 向左移位、向右移位 |
| ~ \| & ^ | 按位取反、按位或、按位与、按位异或 |
| += -= *= /= %= | 赋值运算符，例如 a+=1 相当于 a=a+1，a-=1 相当于 a=a-1 |

**运算命令**　**双小括号 (())**：是 Bash Shell 中专门用来**进行整数运算**的命令。

> 注意：(()) 只能进行整数运算，不能对小数（浮点数）或者字符串进行运算。

> 气象数据多为浮点数（如气温 25.4℃），Bash 原生的 (()) 处理不了。如果需要在 Shell 里算浮点数，我们一般借助外部命令如 bc 或是 awk 配合管道符 | 来完成。

**(())的用法**　((a=10+66))、((b=a-15))、((c=a+b))：这种写法可以在计算完成后给变量赋值。以 ((b=a-15)) 为例，即将 a-15 的运算结果赋值给变量 b。注意，使用变量时不附加 $ 前缀，(()) 会自动解析变量名。
a=$((10+66))、b=$((a-15))、c=$((a+b))：可以在 (()) 前面加上 $ 符号获取 (()) 命令的执行结果，也即获取整个表达式的值。注意，类似 c=((a+b)) 这样的写法是错误的，不加 $ 就不能取得表达式的结果。
((a>7 && b==c))：(()) 也可以进行逻辑运算，在 if 语句中常会使用逻辑运算。
echo $((a+10))：需要立即输出表达式的运算结果时，可以在 (()) 前面加 $ 符号。
((a=3+5, b=a+10))：对多个表达式同时进行计算。

### 3.3.12 IF 语句

**IF 语句**

```
if condition
then
    statement(s)
elif condition2
then
    statement(s)
else
    statement(s)
fi
```

注意，最后必须以 fi 来闭合，fi 就是 if 倒过来拼写。

**文件调试**

| | |
|---|---|
| **-d** | 测试是否为**目录** |
| **-e** | 测试目录或文件**是否存在** |
| -f | 测试是否为文件 |
| -r | 是否有读取权限 |
| -w | 是否有权限写入 |
| -x | 是否有权限执行 |
| -l | 是否为软连接文件 |

示例

```
if [-e ${filename}]
then
echo “文件存在”
fi
```

### 3.3.13 循环语句

**WHILE**

```
while condition
do
    statements
done
```

示例（求和计算）

```
#!/bin/bash        i=1        sum=0
while ((i <= 100))
do
    ((sum += i))
    ((i++)) # i 的值逐步增大
done
echo "The sum is: $sum "       # 运行结果：The sum is: 5050
```

**FOR**

```
for((exp1; exp2; exp3))
do
    statements
done
```

```
for var in list
do
    statements
done
```

（无需等间隔循环）

示例（同样实现 1 到 100 求和）

```
#!/bin/bash     sum=0
for ((i=1; i<=100; i++))
do
    ((sum += i))
done
echo "The sum is: $sum "          # 运行结果：The sum is: 5050
```